

EXP 5-Banker's Deadlock Avoidance Algorithm

[Back To Practice](#)

Banker's Deadlock Avoidance Algorithm
Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.
Input Format
1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Completed

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int main() {
5     int n, m;
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     printf("Enter number of resource types: \n");
10    scanf("%d", &m);
11
12    int Allocation[n][m], Max[n][m], Need[n][m], Available[m];
13
14    printf("Enter Allocation Matrix:\n");
15    for (int i = 0; i < n; i++) {
16        for (int j = 0; j < m; j++) {
17            scanf("%d", &Allocation[i][j]);
18        }
19    }
20
21    printf("Enter Max Matrix:\n");
22    for (int i = 0; i < n; i++) {
23        for (int j = 0; j < m; j++) {
24            scanf("%d", &Max[i][j]);
```

Test Cases

Run Sample Cases

Run All Cases

Select Language : C (GCC 9.2.0)

```
25     }
26 }
27
28 printf("Enter Available Resources: \n");
29 for (int j = 0; j < m; j++) {
30     scanf("%d", &Available[j]);
31 }
32
33 for (int i = 0; i < n; i++) {
34     for (int j = 0; j < m; j++) {
35         Need[i][j] = Max[i][j] - Allocation[i][j];
36     }
37 }
38
39 bool Finish[n];
40 for (int i = 0; i < n; i++) Finish[i] = false;
41
42 int SafeSequence[n];
43 int work[m];
44 for (int j = 0; j < m; j++) work[j] = Available[j];
45
46 int count = 0;
47 while (count < n) {
48     bool found = false;
```

Test Cases 2

Select Language: C (GCC 9.2.0)

```
49     for (int i = 0; i < n; i++) {
50         if (!Finish[i]) {
51             bool possible = true;
52             for (int j = 0; j < m; j++) {
53                 if (Need[i][j] > work[j]) {
54                     possible = false;
55                     break;
56                 }
57             }
58             if (possible) {
59                 for (int j = 0; j < m; j++) {
60                     work[j] += Allocation[i][j];
61                 }
62                 SafeSequence[count++] = i;
63                 Finish[i] = true;
64                 found = true;
65             }
66         }
67     }
68     if (!found) {
69         printf("System is NOT in a safe state (Deadlock may occur).\n");
70         return 0;
71     }
72 }
```

```
74     printf("System is in a SAFE STATE.\n");
75     printf("Safe Sequence: ");
76     for (int i = 0; i < n; i++) {
77         printf("%d ", SafeSequence[i]);
78     }
79     printf("\n");
80
81     return 0;
82 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input:

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output:

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```